

Making Pointers to Members Callable

Contents

- [Revisions](#)
- [Motivation](#)
- [Proposal](#)
- [Impact on Code](#)
- [Potential Concerns](#)
- [Prior Art](#)
- [Acknowledgements](#)

Revisions

Since r0, this paper has focused its motivation much more heavily on generic programming - as it is writing generic code that would benefit most from this feature. Concerns raised about r0 have been addressed in a separate section.

Motivation

Suppose we have a collection of Shapes and want to draw all of them.

```
std::vector<Shape*> shapes;
std::for_each(shapes.begin(), shapes.end(), &Shape::draw); // error
```

That doesn't work. But then later, we try to parallelize this by starting a bunch of threads:

```
std::vector<std::thread> threads;
for (auto shape : shapes) {
    threads.push_back(std::thread(&Shape::draw, shape)); // ok
}
```

The difference between the first fragment failing but the second succeeding is the foundation of this proposal.

Suppose we have a simple type wrapper holding a member of dependent type, and we wish to write a member function that takes a Callable as a template argument and invokes it. The simplest way to implement that would be, setting aside the question of SFINAE:

```
template <class T>
struct Wrapper {
    T val;

    template <class F>
    decltype(auto) call(F f) const {
        return f(val);
    }
};
```

The need to invoke a Callable of dependent type comes up with some regularity. But the clearest, simplest solution to that problem isn't the best. The main problem is with the syntax `f(val)`. That isn't the only syntax to invoke a Callable in C++. In fact, depending on the types of the callable and the first argument, there are **five**:

- `f(t1, t2, ..., tN)`, if `f` is a function, function pointer, or class type with overloaded `operator()`
- `(t1.*f)(t2, ..., tN)`, if `f` is a pointer to member function of a class `t1` derives from
- `((*t1).*f)(t2, ..., tN)`, if `f` is a pointer to member function of a class `*t1` derives from
- `t1.*f`, if `f` is a pointer to member data of a class `t1` derives from
- `(*t1).*f`, if `f` is a pointer to member data of a class `*t1` derives from

In order to allow pointers to members to be used in our Wrapper, one of two steps needs to be taken:

1. We impose on the user the requirement to pass that pointer to member in a way that can work with the `f(val)` syntax, which can either be done with `std::mem_fn()` or a lambda. This approach is taken by all the standard library algorithms and makes all the library code easier to read: all function calls just look like function calls. We just push the awkwardness to

the user, as everybody who has ever tried to write this code would recognize:

```
auto it = std::find_if(widgets.begin(), widgets.end(), &Widget::isFoo);           // error
auto it = std::find_if(widgets.begin(), widgets.end(), std::mem_fn(&Widget::isFoo)); // ok
```

2. We can reimplement `call()` to work with all the different callable types:

```
template <class F>
decltype(auto) call(F f) const {
    return std::ref(f)(val);           // in C++11, though this is likely not used often
    return std::invoke(f, val);        // in C++17
}
```

This makes user code easier to read, but makes the library code more difficult - such code quickly becomes a proliferation of `std::invoke()` s. This can make it harder to answer at a glance the question: which function calls are *passing* the Callable and which are calling it?

This problem gets further complicated when you want to use SFINAE (or eventually Concepts) to constrain your function templates. The ecosystem of the standard library is built around the concept *INVOKE*. All the tools at our disposal (the metafunctions `std::is_invocable`, `std::invoke_result_t` and the concepts like `Invocable`) check if something is *INVOKE*-able. Hence, the following example, despite being common, is subtly incorrect:

```
template <class T>
struct Wrapper {
    T val;

    template <class F>
    std::invoke_result_t<F&, T const&> call(F f) const {
        return f(val);
    }
};
```

It is easy enough to express the constraint that `F` can be directly invoked with `val`, but that's something we have to be vigilant about expressing. Hence, the options at our disposal are more like:

1. Use the `f(val)` in our library throughout, requiring the users to use `std::mem_fn()` for pointers to members, but also write our own type traits and concepts to express these constraints *properly*.
2. Use the standard library's type traits and concepts and use `std::invoke()` everywhere.

The fundamental problem with both choices is: what information do `std::mem_fn()` and `std::invoke()` actually convey to readers of our code?

Arguably, **none**!

Both are only ever used in the context where the Callable type is dependent. After all, if we know we have a function, we would just call the function. If we know we have a pointer to member, we would just use the appropriate syntax to invoke that pointer. These tools are unnecessary in non-dependent contexts. And if the Callable *is* dependent, we may not always need to use `std::invoke()` (e.g. if our Callable is nullary), but it's *never wrong* to use it. These tools become simple annotation: `std::invoke(f, val)` is nothing more than an 11-character annotation for "Here is an invocation of a Callable." `std::mem_fn(ptr)` is likewise just annotation for "Here a pointer to member is being passed to a function template," which is already obvious from context.

Both only exist because we have five syntaxes for invoking Callables. What if we didn't?

Proposal

Pointers to members

Allow the syntax `f(t1, t2, ..., tN)` to work correctly if `f` is a pointer to member function or pointer to member data. That is, let that expression be equivalent to what `INVOKE(f, t1, t2, ..., tN)` currently means. Pointers to members are conceptually functions that take an appropriate class type as their first argument. The language already recognizes this with regards to how the various standard library function objects work (e.g. `std::function`, `std::bind`, `std::thread`). The only thing missing is the syntactic equivalence. This would allow:

```
template <class F>
std::invoke_result_t<F&(T const&> call(F f) const {
    return f(val);
}
```

to be a correctly constrained, fully functional implementation that is easy to read on both the library side (it just looks like a function call) and the user side (we can just pass in a pointer to member without annotation).

This proposal would obviate the need for annotation in all the standard library algorithms as well:

```
vector<Widget*> widgets;
```

```
auto it = find_if(widgets.begin(), widgets.end(), std::mem_fn(&Widget::isFoo)); // ok today
auto it = find_if(widgets.begin(), widgets.end(), &Widget::isFoo);                // error today, ok with proposal
```

C++ is a complicated language, and there is intrinsic value in making the simplest code just simply do the right thing in all cases.

`std::reference_wrapper`

Now, what about `std::reference_wrapper`? This proposal started by defining five syntaxes for invoking callables, but according to the standard's definition of *INVOKE* in [func.require], there are actually seven - two of which treat `std::reference_wrapper` as special. Would those overloads still need to be special? If pointers to members become callable, then the mental model for thinking about them would move towards thinking of them as functions. And if they're just functions, we can redefine how they're invoked to align with other functions. Consider:

```
struct X {
    bool isFoo() const;
};

auto f = &X::isFoo;
f(x);
```

For what expressions of `x` should that expression be well-formed? The paper proposes that the model for resolution of this call should consider `f` as if it were an overloaded function like:

```
bool f_impl(X const* x) { return (x->*f)(); } // #1
bool f_impl(X const& x) { return (x.*f)(); } // #2
```

This overloaded function approach would already correctly handle pointers to `x` (#1) or objects of type `x` (#2) or objects of type `std::reference_wrapper<X>` (#2 by way of operator `X&()`), or any types that inherit from `x` in similar ways. But there is one group of types that still doesn't work: proxy pointers. We would just need to properly define a rule to address these types, and ensuring that `X* operator->` would have to take precedence over operator `X&()`. One such model could be:

```
bool f_impl(X const* x) { return (x->*f)(); } // #1
bool f_impl(X const& x) { return (x.*f)(); } // #2

template <class P>
auto f_impl(P& p) -> decltype((p->*f)()) { return (p->*f)(); } // #3
```

This proposal is based on the premise that define this new model for invoking pointers to members greatly simplifies the everyday act of writing code in a way that far outstrips the added complexity of defining these rules themselves. Arguably, these rules are no more complicated than *INVOKE* is today.

Impact on Code

This proposal would make it easier to write generic code that is more functional. There would be no more need to have to think about pointers to members as a special category of Callable. There would simply be Callables, and not Callables. It would make `std::invoke()` and `std::mem_fn()` obsolete. It would also remove a source of error in writing SFINAE-friendly generic code by using `std::invoke_result` or `std::is_invocable` without `std::invoke()`.

This proposal would break code that exists today. One potential implementation of `std::invoke()` would be to simply have five overloads for the five different syntaxes that are disambiguated with expression SFINAE. There is currently no overlap between the five syntaxes: no set of Callable and arguments is valid for more than one. But this proposal would make `f(t1, t2, ..., tN)` valid for all of them, making all the calls to such an implementation ambiguous for pointers to members. Such code would be easy to fix - simply don't use `invoke()`.

Potential Concerns

There were a few concerns brought up when this proposal was presented in Oulu.

Does this proposal affect overload resolution?

This proposal does not change the rules of overload resolution. It only seeks the change the meaning of the syntax `f(val)` where `f` is a pointer to member. Name lookup finds `f` as normal, and if the result is a pointer to member, it will be a pointer to a specific member function or specific member data. At that point, there is no overload resolution to perform, save to determine if the function call is viable.

Is this proposal related to Unified Call Syntax [1]?

This proposal does not change the rules of name lookup and is not concerned with the *names* of member functions, only pointers to them. Unified Call Syntax deals with name lookup, but in the context of a dependent function call, the same limitation on

having multiple invocation syntaxes still applies.

This proposal would allow automatic dereferencing of pointers in the language

While pointers to functions are automatically dereferenced when invoked, function arguments are not, so this would certainly be new, and potentially dangerous. Consider:

```
struct Widget {
    int foo();
};

int bar(Widget* pw, int (Widget::*f)()) {
    return f(pw); // pw dereferenced, no annotation
}
```

Yes, this proposal would be the first place in the language where an argument to a call could be automatically dereferenced without any language annotation. But this paper believes this concern is unwarranted. Today, pointers to members will either be passed through `std::mem_fn()` or through `std::invoke()` (or through `std::bind()` or `std::thread` or `std::ref()` or `std::function`), all of which will automatically dereference their first argument if it's a pointer. While these dereferences happen in *code* and not in the language, that code is buried in the standard library implementations of these tools which isn't something people look at on a day to day basis. This paper believes that this is a distinction without a practical difference, and the utility of being able to write function calls that invoke all the things outweighs the potential danger of hidden pointer dereferencing.

Prior Art

This idea was previously proposed by Peter Dimov in N1695 [2]. This proposal predates the committee recording discussions and the precise reason why it was rejected is unclear. This paper believes that paper was a good idea thirteen years ago and continues to be a good idea today.

Acknowledgements

Thanks to Jonathan Wakely, Matt Calabrese, and Peter Dimov for the encouragement, support, and review of the paper.

[1] [Wording for Unified Call Syntax](#)

[2] [A Proposal to Make Pointers to Members Callable](#)